
Linux Cron Tab: A Beginner's Guide

Cron is a **time-based job scheduler in Unix-like systems**. It allows users and administrators to run scripts or commands automatically at specified intervals (minutes, hours, days, etc.).

1. Cron Concepts

- **Cron daemon** ([cron](#)) runs in the background and executes scheduled jobs.
 - **Cron job**: a task scheduled in cron.
 - **Crontab**: the configuration file where cron jobs are defined.
 - Each user can have their own crontab.
 - There's also a system-wide crontab at [/etc/crontab](#).
-

2. Cron Service on Ubuntu

Ubuntu uses [cron](#), not [crond](#).

Check if cron is running:

```
systemctl status cron
```

Start cron if it's not running:

```
sudo systemctl start cron
```

```
sudo systemctl enable cron # starts cron on boot
```

3. Crontab Syntax

A crontab line looks like this:

```
* * * * * /path/to/command arg1 arg2
```

The five asterisks represent **time and date fields**:

Field	Value s	Description
Minute	0-59	Minute of the hour
Hour	0-23	Hour of the day
Day of Month	1-31	Day of the month
Month	1-12	Month of the year
Day of Week	0-7	Day of the week (0 or 7 = Sunday)

Example:

```
30 2 * * 1 /home/ubuntu/backup.sh
```

- Runs `/home/ubuntu/backup.sh` every Monday at 2:30 AM

4. Common Cron Patterns

Schedule	Example
Every minute	<code>* * * * * command</code>
Every 5 minutes	<code>* /5 * * * * command</code>

Hourly	<code>0 * * * * command</code>
Daily at midnight	<code>0 0 * * * command</code>
Every Sunday at 3 AM	<code>0 3 * * 0 command</code>

5. Editing Your Crontab

To edit your user crontab:

```
crontab -e
```

- Opens the crontab in your default text editor.
- Save and exit to apply changes.

To list your cron jobs:

```
crontab -l
```

To remove your crontab:

```
crontab -r
```

6. System-Wide Cron Files

- `/etc/crontab` – system-wide cron jobs
- `/etc/cron.d/` – additional system jobs
- `/etc/cron.hourly/`, `/etc/cron.daily/`, `/etc/cron.weekly/`, `/etc/cron.monthly/` – scripts placed here run automatically on that schedule

Example (`/etc/crontab`):

```
SHELL=/bin/bash
```

```
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
```

```
# m h dom mon dow user  command
```

```
0 4 * * * root /usr/local/bin/cleanup.sh
```

7. Cron Best Practices

1. **Use full paths** for scripts and commands.

Redirect output to log files for debugging:

```
* * * * * /home/ubuntu/script.sh >> /home/ubuntu/script.log 2>&1
```

- 2.
3. **Test scripts manually** before adding to cron.
4. Avoid running multiple heavy tasks at the same time.

Use comments in crontab for clarity:

```
# Run backup every day at 2 AM
```

```
0 2 * * * /home/ubuntu/backup.sh
```

- 5.
-

8. Troubleshooting Cron Jobs

Check cron logs:

```
grep CRON /var/log/syslog
```

- - Ensure the cron daemon is running.
 - Verify permissions and ownership of scripts.
 - Always use **absolute paths** for commands and files.
-

9. Quick Example: Daily Backup

1. Create a script `/home/ubuntu/backup.sh`:

```
#!/bin/bash
```

```
tar -czf /home/ubuntu/backup_$(date +%F).tar.gz /home/ubuntu/data
```

2. Make it executable:

```
chmod +x /home/ubuntu/backup.sh
```

3. Add to crontab:

```
crontab -e
```

Add line:

```
0 2 * * * /home/ubuntu/backup.sh >> /home/ubuntu/backup.log 2>&1
```

- Runs every day at 2:00 AM and logs output.
-